SA Time Stability Tools 0.0.2 — Time Stability App

Python toolkit and desktop GUI for long-time-measurement frequency stability and phase noise. It computes NIST SP 1065 time- and frequency-domain statistics, derives SSB phase noise, classifies power-law noise types, converts vendor spectrum-analyzer exports, and writes Stable32 files and native Veusz HDF5 (`.vszh5`) documents.

Version	**0.0.2** (Semantic Versioning 2.0.0 [21]; `0.0.x` = internal draft)
Date	2026-10-11
Author	William W. Wallace · NRAO/GBO · wwallace@nrao.edu
Licence	GPL-3.0-or-later (SPDX); full text in `LICENSE`
GUI	`Time_Stability_App.py` (was `gui_app.py` in 0.0.1)
Python	3.8 – 3.13. Test suite run on 3.8.20, 3.12.13 and 3.13.16
Veusz	Not needed to write the `.vszh5`; checked against Veusz 3.6.2 and 4.2.1
Status	Internal draft, validated on synthetic data only

Changes are listed in `CHANGELOG.md`. **0.0.2 is pass 1 of 2:** it changes only the Veusz output (plus the GUI rename, file headers and Spyder cells). Defects found in review are listed under Known issues and are fixed in 0.0.3.

What changed in 0.0.2

Item	0.0.1	0.0.2
Veusz file	`veusz.embed` + `Embedded.Save()`, which defaults to `mode='vsz'`, so it wrote a **text** `.vsz` under the `.vszh5` name. Veusz opens `.vszh5` with its HDF5 loader → *file signature not found*. Data kept to 7 significant digits	**Native HDF5** `.vszh5` written with h5py through `vszh5_writer.py` 0.0.5 (from `isp_speed_monitor.py` 1.3.0, shared with QuickSyn) [17]. Data at float64; opens in Veusz with *File › Open*
Veusz needed?	Yes, to write (PyQt6 + qmake source build)	**No** to write; optional to render PDF/PNG
Content	3 pages: raw/residual, sigma-tau (TDEV on the same axes as ADEV), L(f)	Every result in tagged datasets; 6 pages: overview, phase, y(t), sigma-tau, **TDEV on its own axes**, L(f)
Process isolation	Export ran in a subprocess (PyQt6 vs PySide6)	Export runs in-process (no Qt imported); only the optional render runs in a child process
GUI	`gui_app.py`	`Time_Stability_App.py`, title "Time Stability App 0.0.2"
Headers	none	Full header with version in **every** `.py`; Spyder `# %%` cells

Contents

File	Version	Purpose
`Time_Stability_App.py`	0.0.2	PySide6 GUI tying all modules together
`stability_core.py`	0.0.2	Canonical CSV I/O, drift fitting, ADEV/OADEV/MDEV/TDEV/OHDEV/Thêo1/ThêoBR/ThêoH, time-domain noise ID
`phase_noise.py`	0.0.2	SSB phase noise L(f) via Welch PSD, direct-trace parsing, frequency-domain noise ID, L(f)↔S_y(f)

File	Version	Purpose
`sa_convert.py`	0.0.2	Keysight / R&S / generic SA export → canonical CSV; batch marker/peak extraction
`stable32_export.py`	0.0.2	Stable32-compatible ASCII `.PHD` / `.FRD` / `.DAT` writer
`veusz_export.py`	0.0.2	Native Veusz HDF5 export (`export_vszh5`) and optional render (`render_vszh5`, CLI `--render`)
`vszh5_writer.py`	0.0.5	Shared Veusz HDF5 writer, byte-identical to the QuickSyn 0.0.5 copy (SHA-256 `2cdf43d2…ae33c1c`)
`validate_core.py`	0.0.2	Estimators vs allantools, noise slopes, Thêo1/ThêoH
`validate_phase_noise.py`	0.0.2	L(f) slopes of synthetic power-law noise
`validate_sa_convert.py`	0.0.2	Converter on synthetic vendor-style files
`validate_veusz_export.py`	0.0.2	`.vszh5` layout and bit-exact data, without Veusz
`validate_vszh5_in_veusz.py`	0.0.2	Round trip through Veusz's own loader and renderer (needs Veusz)
`validate_gui.py`	0.0.2	Headless GUI smoke test through the window's own export methods
`environment.yml`	0.0.2	mamba/conda-forge environment
`requirements.txt`	0.0.2	pip alternative
`CHANGELOG.md`, `LICENSE`	—	Change history; GPL-3.0 text
`README.pdf`	0.0.2	This README rendered to PDF (replaces the 0.0.1 `SA Time Stability Tools-….pdf`)
`example_data/example_master_phase.csv`	—	Example record (8193 points, `time_s,phase_s`). The values are synthetic in scale (x of order 10^3 s), so L(f) comes out far above 0 dBc/Hz — use it for workflow checks only

Installation (mamba, conda-forge)

```
mamba env create -f environment.yml
mamba activate sa_stability
python Time_Stability_App.py
```

`h5py` writes the `.vszh5`; Veusz is **not** required for that. Veusz is in the environment so you can open the documents and render PDF/PNG; remove it from `environment.yml` if you do not want it. A standalone Veusz (Windows installer) works as well: point `TIME_STABILITY_VEUSZ` at `veusz.exe`.

pip alternative: `python -m pip install -r requirements.txt` (see the comments in that file; the PySide6 pin there excludes Python 3.13, known issue 12).

Running the GUI

```
python Time_Stability_App.py
```

Workflow: **Load Canonical CSV** (or **Convert Vendor SA File**) → check `tau0` (auto-filled from the time stamps) and `f0` (nominal carrier, used to scale phase noise) → **Remove Drift** (optional, order 0/1/2) → **Run Stability Analysis** and/or **Run SSB Phase Noise** → **Export Stable32 File** / **Export Veusz**

(.vszh5). With **Render PDF/PNG** ticked, a multipage PDF and one PNG per page are rendered next to the `.vszh5` in the background; the log reports the files or the reason it was not done.

Spyder

Every file is divided into `# %%` cells (header, imports, one cell per section, entry point, a commented scratch cell) and runs cell by cell in order (checked with a harness that reproduces Spyder's `runcell` namespace handling). The scripts capture their own path once (Spyder defines `__file__` only while a cell runs), never call `sys.exit()` inside IPython, and the GUI reuses an existing `QApplication`. With the console's graphics backend set to Qt the window stays live and `main()` returns at once; with the inline backend `main()` runs the event loop until the window closes.

Canonical CSV format

Two-column CSV with a header naming the value column:

```
time_s,phase_s        # phase / time-error data (seconds), OR
time_s,freq_hz        # absolute frequency readings (Hz), OR
time_s,frac_freq      # fractional frequency y = (f - f0)/f0 (dimensionless)
```

`time_s` must increase monotonically; near-uniform spacing is assumed for the ADEV-family estimators (`tau0` is the median spacing). `stability_core.load_canonical_csv()` converts a frequency column to phase itself (`freq_hz` is first normalised by its own median) and always returns the kind `'phase'`. With a frequency column the phase has one more sample than the time stamps (known issue 1).

Vendor spectrum-analyzer conversion — important caveat

Exact SA export layouts vary by instrument model and firmware revision. `sa_convert.py` implements:

- a **generic fallback parser** that detects the delimiter (`,` / `;` /tab), skips metadata/comment lines, and classifies column 0 as a time series (near-uniform spacing) or a frequency sweep/trace (log-uniform spacing, large span, no zero values) — validated on synthetic files (`validate_sa_convert.py`);
- **Keysight**- and **R&S**-flavoured parsers that take the first numeric data block after a free-text/metadata header, since neither vendor uses one fixed header format across firmware versions and product lines (PXA/MXA/N-series vs FSW/FSVA/FSWP).

Before trusting a bulk conversion on real instrument data, export one small sample file from your instrument and firmware and confirm the value units (Hz vs dBc/Hz vs dBm, phase vs frequency) match what the converter classified. Extend `_VENDOR_PARSERS` in `sa_convert.py` for another layout. Known limits in 0.0.2: decimal-comma exports are not read (issue 4) and the canonical CSV keeps 11 significant digits (issue 5).

`batch_extract_marker_time_series()` builds a series from repeated single-sweep snapshots (peak/marker tracking). It assumes uniform capture spacing; map the returned index to elapsed time from your own capture time stamps.

Stable32 export format

Implements the ASCII data-file convention of the Stable32 manual [12]: plain text, tab-delimited `timetag value` pairs, exponential notation, and the "gap = 0" convention (a missing point is written as `0`; a gap at the first or last point raises `ValueError`). A `#`-prefixed header block (File/Date/Type/Title/Columns) is written for documentation; check that your Stable32 version tolerates comment lines if strict compliance matters.

Veusz export (`.vszh5`)

`veusz_export.export_vszh5()` writes one native Veusz HDF5 document with h5py through `vszh5_writer.py` [17]. No Veusz and no Qt binding is imported, so the GUI calls it in-process. The write is atomic: the file is written to `<name>.vszh5.part` and moved into place, so a failed export leaves the previous file intact. A name without an HDF5 extension gets `.vszh5` appended, because Veusz picks its loader from the extension.

Datasets (float64; Veusz tag in brackets):

Dataset	Content	Tag
`raw/time_s`, `raw/phase_s`	Input x(t)	raw
`raw/residual_s`	Drift-removed x(t)	raw
`raw/freq_time_s`, `raw/frac_freq`	$y_k = (x_{k+1} - x_k) / \tau_0$ at interval midpoints	raw
`raw/frac_freq_residual`	y(t) of the residual	raw
`stability/tau_s, m, n_used, adev, oadev, mdev, ohdev, tdev_s`	AVAR-family sweep	stability
`stability/theoh_tau_s, theoh_dev, theoh_m, theoh_source`	ThêoH curve and the estimator of each point	stability
`noise_id/*`, `phase_noise_id/*`	Time- and frequency-domain noise-ID tables	noise_id
`phase_noise/freq_hz`, `phase_noise/l_dbc_hz`	SSB L(f)	phase_noise
`curves/*`	Extra curves passed as `sigma_tau_series`	curves
`meta/title, source, generator, parameters_json, datasets`	Run description, tau0, f0, drift fit	meta

Pages: `page_overview` (2 × 2), `page_phase`, `page_frequency`, `page_sigma_tau` (ADEV, OADEV, MDEV, OHDEV, ThêoH), `page_tdev` (seconds, separate from the dimensionless deviations), `page_phase_noise`. Pages without data are left out.

Rendering (optional). `render_vszh5()` / `python veusz_export.py --render FILE.vszh5` renders one PDF with every page and one PNG per page. `veusz --export` cannot open a `.vszh5`, so the Veusz program is driven in `--listen` mode with Veusz's HDF5 loader, with the veusz Python package as the fallback [17]. The program is searched in `TIME_STABILITY_VEUSZ`, then `VEUSZ_EXE`, then `PATH` and the usual install folders. The GUI renders in a child process so Veusz's own Qt binding never loads next to PySide6.

`render_vszh5()` also works around three limits of `vszh5_writer.render_pages()` 0.0.5 found here (reported for the next writer release): the PDF of `run v0.0.2.vszh5` was named `run v0.0.pdf` (`with_suffix`); a re-render listed no files (it reports only new files); and `png_pages="first"` wrote no PNG with Veusz 4.2.1, whose command namespace has NumPy's `min` (`min(1, n)` → `numpy.min(1, axis=n)` → `AxisError`, swallowed in `--listen` mode). Missing outputs are now reported as an error.

The 0.0.1 functions `export_all_plots()` and `export_all_plots_subprocess()` remain as wrappers and now write real HDF5.

Time-domain noise classification — reference table

Noise type	ADEV slope μ	MDEV slope μ
White PM	−1.0	−1.5
Flicker PM	−1.0	−1.0
White FM	−0.5	−0.5

Noise type	ADEV slope μ	MDEV slope μ
Flicker FM	0.0	0.0
Random Walk FM	+0.5	+0.5
Flicker Walk FM	+1.0	+1.0
Linear Frequency Drift*	+1.5	+1.5

(`stability_core.NOISE_TABLE`, per [1] Sec. 3.4, [2].) *As coded in 0.0.2; this row is wrong — slope +1.5 is Random-Run FM, and linear frequency drift gives slope +1 (known issue 7).

Frequency-domain (SSB phase noise) classification — reference table

Noise type	L(f) slope (dB/decade)
White PM	0
Flicker PM	−10
White FM	−20
Flicker FM	−30
Random Walk FM	−40

(`phase_noise.FREQ_NOISE_TABLE`, per [1] Table 1 / [11] Ch. 2.) Because $\mathcal{L}(f) = 10\log_{10}[\frac{1}{2}S_\phi(f)]$ is already in dB, its slope against $\log_{10} f$ is 10× the $S_\phi(f)$ power-law exponent. `validate_phase_noise.py` measures −19.2/−29.2/−39.2 dB/decade against −20/−30/−40. The three cases share one random realisation (seed 17), so the common −0.8 offset is that realisation's error, not an estimator bias. `classify_freq_domain_noise()` log-bin-averages L(f) before taking slopes; treat single adjacent-bin classifications as a noisy trend and read the trend across a full decade.

Equation appendix

Equations use `$$ … $$` and `$ … $` so they render in Obsidian.

Overlapping Allan deviation (OADEV), phase form [1]:

$$\sigma_y^2(m\tau_0) = \frac{1}{2(m\tau_0)^2(N-2m)} \sum_{i=1}^{N-2m} \left(x_{i+2m} - 2x_{i+m} + x_i \right)^2$$

Modified Allan deviation (MDEV), phase form [1]:

$$\mathrm{Mod}\,\sigma_y^2(m\tau_0) = \frac{1}{2m^2(m\tau_0)^2(N-3m+1)} \sum_{j=1}^{N-3m+1} \left[\sum_{i=j}^{j+m-1} \left(x_{i+2m} - 2x_{i+m} + x_i \right) \right]^2$$

Time deviation (TDEV) [1]:

$$\sigma_x(\tau) = \frac{\tau}{\sqrt{3}}\,\mathrm{Mod}\,\sigma_y(\tau)$$

Overlapping Hadamard deviation (OHDEV), phase form [1]:

$$\sigma_H^2(m\tau_0) = \frac{1}{6(m\tau_0)^2(N-3m)} \sum_{i=1}^{N-3m} \left(x_{i+3m} - 3x_{i+2m} + 3x_{i+m} - x_i \right)^2$$

Thêo1 ([1] Eq. 30, [5]); m even, $10 \le m \le N-1$. With $w = m\,/\,2 - \delta$ and 0-based i:

$$\mathrm{Theo1}(m, \tau_0, N) = \frac{1}{0.75\,(N-m)\,(m\tau_0)^2} \sum_{i=0}^{N-m-1} \sum_{w=1}^{m/2} \frac{\left[(x_i - x_{i+w}) + (x_{i+m} - x_{i+m-w}) \right]^2}{w}$$

with effective averaging time

$$\tau_{\text{eff}} = 0.75\, m\, \tau_0 \quad (\text{not } m\tau_0)\,.$$

ThêoBR / ThêoH bias correction (Howe ladder-ratio method [4], [6]):

$$G = \frac{1}{n+1}\sum_{i=0}^{n}\frac{\sigma_y^2(m=9+3i)}{\text{Theo1}(m=12+4i)}, \qquad n = \left\lfloor \frac{0.1N}{3} - 3 \right\rfloor, \qquad \widehat{\text{ThêoBR}}(m) = \sqrt{G}\cdot\text{Theo1}(m)$$

evaluated at $\tau = 0.75\, m\, \tau_0$. ThêoH uses AVAR for $m \le k = 0.1N$ and ThêoBR from $m = k\,/\,0.75$ upward. The code averages at most 40 evenly spaced ladder rungs.

SSB phase noise [1], [11]:

$$\phi(t) = 2\pi f_0\, x(t), \qquad \mathcal{L}(f) = 10\log_{10}\left[\tfrac{1}{2}\,S_\phi(f)\right] \ \text{dBc / Hz}$$

where $S_\phi(f)$ is the one-sided (Welch [10]) power spectral density of $\phi(t)$.

L(f) ↔ fractional-frequency PSD [1]:

$$S_\phi(f) = 2\cdot 10^{\mathcal{L}(f)\,/\,10}, \qquad S_y(f) = \frac{f^2}{f_0^2}\,S_\phi(f)$$

Fractional frequency from phase (as `phase_to_freq()` and the `.vszh5`):

$$y_k = \frac{x_{k+1} - x_k}{\tau_0}$$

Polynomial drift removal (least squares on $x(t)$; the residual $\varepsilon(t)$ feeds every estimator above):

$$x(t) = a_0 + a_1 t + a_2 t^2 + \varepsilon(t), \qquad D = \frac{dy}{dt} = 2a_2 \quad [\text{s}^{-1}]$$

Validation performed (0.0.2)

Run in this session on Python 3.8.20 (NumPy 1.24.4, SciPy 1.10.1, PySide6 6.6.3.1), 3.12.13 and 3.13.16 (NumPy 2.5, PySide6 6.12); Veusz 3.6.2 (Ubuntu) and 4.2.1 (built from the PyPI source).

Script	Result
`validate_core.py`	OADEV/MDEV/OHDEV/TDEV = allantools to 0.000000 %; Thêo1/OADEV(τ_eff) = 1.000, 0.998, 0.988, 0.971 (m = 20…200, white FM); ThêoH reaches 62.8 % of the record (OADEV 50 %); ADEV slopes of 5 noise types within 0.11 of the table
`validate_phase_noise.py`	L(f) slopes −19.20/−29.20/−39.20 dB/decade vs −20/−30/−40; S_y(f) finite and positive
`validate_sa_convert.py`	Generic, Keysight-style, R&S-style files classified as expected; batch peaks ≈ −50 dBm
`validate_veusz_export.py`	21/21: HDF5 Veusz format 1, every array bit-exact, script compiles, 6 pages, title escaping, 0.0.1 wrappers write HDF5, failed export leaves the old file
`validate_vszh5_in_veusz.py`	12/12 with Veusz 3.6.2 and 4.2.1: Veusz's loader, all datasets bit-exact after load, types and tags, every plot reference resolves, in-process PDF + PNG of every page, `--listen` program render, re-render
`validate_gui.py`	Load, drift, stability, phase noise, Stable32 and `.vszh5` export through the window's methods, background render: 7 files
Spyder cells	All 13 `.py` files run cell by cell (Spyder `runcell` namespace rules) from an unrelated working folder
0.0.1 file	Confirmed text, not HDF5; Veusz 3.6.2 HDF5 loader: *Unable to open file (file signature not found)*

Not verified: Windows (paths, the Veusz installer's `--listen` behaviour), a real `mamba env create`, instrument data, Stable32 import of the files.

Known issues (fixed in 0.0.3)

1. A canonical CSV with `freq_hz` or `frac_freq` loads with `len(x) = len(t) + 1`. Drift removal, plotting and export then fail.
2. Loading a new file keeps the previous drift fit and results, so a stale residual is applied to the new data.
3. The GUI logs "converting to phase" on every load (the kind returned is always `'phase'`).
4. Semicolon exports with decimal commas (typical R&S locale) parse to "no numeric data".
5. `convert_to_canonical_csv()` keeps 11 significant digits: a 10 GHz series loses 3.7 mHz, more than its 1.2 mHz steps.
6. The Stable32 writer leaves a partial file when it rejects a gap at the last point.
7. `NOISE_TABLE` row "Linear Freq. Drift" (+1.5) is Random-Run FM; linear drift gives +1 and is reported as Flicker-Walk FM.
8. The drift-rate log line says "s/s^2"; $D = 2a_2$ has units of 1/s.
9. All analysis runs on the GUI thread; no parallel or GPU path.
10. Several references below are garbled or unverified.
11. `validate_core/phase_noise/sa_convert.py` print PASS/FAIL but always exit 0.
12. `requirements.txt` pins `PySide6~=6.6.3`, which has no Python 3.13 wheels.
13. The GUI uses PySide6 directly (not Qt.py) and has no menu bar or light/dark display option.

IEEE reference table

As received in 0.0.1, plus [17]–[21]; checked and corrected in 0.0.3.

#	Reference	URL
[1]	W. J. Riley, "Handbook of Frequency Stability Analysis," NIST Special Publication 1065, Jul. 2008.	https://tf.nist.gov/general/pdf/2220.pdf
[2]	J. E. Kamas and S. R. Stein, Eds. (orig. D. A. Howe, D. W. Allan, J. A. Barnes), "Characterization of Clocks and Oscillators," NIST Special Publication 559 / SP1065 predecessor; see also D. W. Allan, "Time and Frequency (Time-Domain) Characterization, Estimation, and Prediction of Precision Clocks and Oscillators," NIST, 1990. *(as received; corrected in 0.0.3)*	https://tf.nist.gov/general/pdf/461.pdf
[3]	"IEEE Standard Definitions of Physical Quantities for Fundamental Frequency and Time Metrology — Random Instabilities," IEEE Std 1139-2008.	https://doi.org/10.1109/IEEESTD.2008.4797525
[4]	D. A. Howe, "ThêoH: A Hybrid, High-Confidence Statistic that Improves on the Allan Deviation," *Metrologia*, vol. 43, no. 4, pp. S322–S327, 2006.	https://doi.org/10.1088/0026-1394/43/4/S17
[5]	D. A. Howe and F. Vernotte (T. N. Tasset), "Thêo1: Characterization of Very Long-Term Frequency Stability," Proc. 18th EFTF, 2004. *(as received; corrected in 0.0.3)*	https://tf.nist.gov/general/pdf/1894.pdf
[6]	Purdue University OxideMEMS Lab, "ThêoH Bias Removal," Proc. 2006 IEEE IFCS. *(as received; corrected in 0.0.3)*	https://engineering.purdue.edu/oxidemems/conferences/fcs2006/PDFs/Papers/145_6180.pdf
[7]	D. A. Howe and T. N. Tasset (as [5]) — corrected Theo1 closed form used in `stability_core.theo1()`.	—
[8]	IEEE Std 1139-2008 (as [3]) and NIST SP1065 [1] Eq. 30 — Theo1 effective-tau relation.	—
[9]	"Allantools Documentation."	https://allantools.readthedocs.io/en/latest/functions.html
[10]	P. Welch, "The Use of Fast Fourier Transform for the Estimation of Power Spectra," *IEEE Trans. Audio Electroacoust.*, vol. 15, no. 2, pp. 70–73, Jun. 1967.	https://doi.org/10.1109/TAU.1967.1161901

#	Reference	URL
[11]	E. Rubiola, *Phase Noise and Frequency Stability in Oscillators*. Cambridge Univ. Press, 2008.	https://doi.org/10.1017/CBO9780511812798
[12]	"Stable32 Software Manual," Hamilton Technical Services, rev. 1.54.	http://www.stable32.com/Manual154.pdf
[13]	D. A. Howe, "ThêoH ladder-ratio bias-factor method" (as [4], [6]).	—
[14]	J. A. Barnes *et al.*, "Characterization of Frequency Stability," *IEEE Trans. Instrum. Meas.*, vol. IM-20, no. 2, pp. 105–120, May 1971.	—
[15]	Keysight Technologies, "PXA/MXA Signal Analyzer Phase Noise Measurement Application User's Guide."	—
[16]	Rohde & Schwarz, "R&S FSWP Phase Noise Analyzer and VCO Tester User Manual."	—
[17]	W. W. Wallace, `vszh5_writer.py` 0.0.5 (from `isp_speed_monitor.py` 1.3.0, Adaptive Electrodynamics LLC dba AdaptEd; shipped with QuickSyn 0.0.5), 2026.	Supplied by the author (not public)
[18]	Veusz Developers, "Veusz — a scientific plotting package," and Veusz 4.2.1 source (`document/doc.py`, `document/loader.py`, `document/commandinterface.py`, `windows/mainwindow.py`).	https://veusz.github.io/ ; https://pypi.org/project/veusz/4.2.1/
[19]	A. Collette *et al.*, "h5py."	https://pypi.org/project/h5py/
[20]	"PySide6 Documentation," Qt for Python; J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.	https://doc.qt.io/qtforpython-6/ ; https://doi.org/10.1109/MCSE.2007.55
[21]	T. Preston-Werner, "Semantic Versioning 2.0.0."	https://semver.org/spec/v2.0.0.html